



BEGINNERS GUIDE TO TDD (TEST DRIVEN DEVELOPMENT)

TEST DRIVEN DEVELOPMENT

TDD is one of only a few techniques that help us to design better software, faster.

In order to create software that is easily “testable” we need software that we can interact with and that allows us to capture the results of those interactions so that we can match them in the assertions in our tests.

“Testable” code is more modular, more loosely-coupled, has better separation-of-concerns, is better abstracted and is more highly-cohesive. These are also the hallmarks of well-designed software. So TDD helps to steer us towards better design.

This is largely as a result of TDD forcing us to apply Dependency Inversion so that we can inject test-time components with which we can capture, and fake, those interactions that we want to test.

TDD is less about testing and more about good design.

RED > GREEN > REFACTOR

RED

Start with a failing test.

We always write the test BEFORE we write the code. See the test fail before moving on to the next step. Create a mini executable specification. This is expressed as a specification of the behavioural intent of the code, i.e what the code should do. So we start each test with the word SHOULD.

Your test should focus only on WHAT the system/code should do; and say nothing about HOW it does it.

Write all tests from the USER PERSPECTIVE – note that the user of your code may be another programmer, not just an end-user.

When you write your test, PREDICT what you think the outcome will be – this helps in the understanding and accuracy of the test, and understanding what's going on if you get an unexpected result.

LISTEN to the test. If the test is difficult to set up it is telling you that your code may be too complex, have poor separation of concerns, be tightly coupled and difficult to debug. Use the signals from your tests to make decisions to improve the quality of your code and design.

GREEN

Make the smallest possible changes to your code to get from a failing test to a passing test. Don't try and change, or improve, everything at this stage. Think of yourself being in a less safe place (a failing test), and your goal is to get back to "safety" (a passing test) as quickly and simply as you can. Use your tools, like refactoring IDEs to help you change the code in small simple steps.

REFACTOR

Work in SMALL STEPS. Improve abstraction, delineate different attributes and behaviours of the code/system, to create boundaries in your design. Use these "boundaries" to keep the different concerns in your software separate.

"Create a tiny universe where the software exists to do one thing, and do it well" – Josh Graham.

TOP TIPS

Simulate the inputs to your system and ALWAYS use fake dependencies. Use collaborators at points of measurement (PoM), and inject fake PoMs to replace the dependencies.

A high level of test coverage is not a useful goal in itself, but may be a side-benefit of a comprehensive approach to TDD. Don't chase coverage, instead drive changes to the code from tests.

Don't try to write lots of tests before you start coding, that is not TDD. Instead write the simplest test that you can think of, and see it fail. Write just enough code to make it pass and then refactor code and test to make them great. Then add the next test.

TDD is about evolving our solutions and designs as a series of small steps.

AUTOMATE nearly everything. No manual units tests.

Getting Going with TDD

Learning TDD is a skill and takes time. There are some good online resources to help.

Cyber Dojo allows you to practice TDD in a variety of languages, with a variety of test frameworks to solve a variety of simple problems (coding Katas). It is free, so give it a try.

Start by dedicating 30 minutes every day for at least 2 weeks to practice "Coding Katas". It is OK, even good, to repeat the same Kata many times. Employ the TDD discipline rigorously to drive the development of the Kata. Solving the problem in a Kata is NOT the goal, practicing the discipline is the goal.

So, RED, GREEN, REFACTOR!